

Supplementary Material: A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

This supplement gives the full proofs of the two theorems stated in the main paper, together with the implementation-effort plot referenced in the Results. It is self-contained: we first restate the soft/hard formulation, then prove the forward-equivalence and temperature-limit results, and finally show the effort timeline.

1 Setup and Notation

The differentiable emulator runs one of two transition maps over the full VCS state $x = (\text{registers}, \text{RAM}, \text{RIOT}, \text{TIA}, \text{frame buffer})$: the bit-exact *hard* map Φ_H and the differentiable *soft* map Φ_S . Both are the per-opcode handler selected by the byte at the program counter (pc). Four primitives appear in the handlers.

One-hot reads. A ROM or RAM read at integer address a is the dot product

$$\text{peek}(r, a) = \mathbf{1}_a^\top r = \sum_i \llbracket i = a \rrbracket r_i = r_a. \quad (1)$$

Softmax dispatch. With a score vector (logits) $\ell \in \mathbb{R}^K$ over the $K = 256$ opcodes, temperature $T > 0$, and softmax weights $w = \text{softmax}(\ell/T)$, the relaxed dispatch output over candidate handler rows V is

$$\text{select}(\ell, V; T) = w^\top V, \quad w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}}. \quad (2)$$

Sigmoid branch. For a relaxed status flag $f \in [0, 1]$ written as a logit $z = s(2f - 1)$ (sign s for branch-when-set/clear) and sharpness α , the gate and relaxed program counter are

$$g = \sigma(\alpha z), \quad \text{pc}_{\text{soft}} = (1 - g) \text{pc}_{\bar{b}} + g \text{pc}_b. \quad (3)$$

Straight-through estimator. For a soft value with a matching hard value,

$$\text{STE}(\text{soft}, \text{hard}) = \text{soft} + \text{sg}(\text{hard} - \text{soft}), \quad (4)$$

whose forward value is *hard* and whose gradient is that of soft.

In the executed soft step, dispatch uses the saturated (hard) form of (2), and the branch returns $\text{STE}(\text{pc}_{\text{soft}}, \text{pc}_{\text{hard}})$; (2) and (3) in their relaxed form define the “fully relaxed” variant analysed in Theorem 2.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Assumption (DM, decision margins). Along the executed trajectory of length N , every discrete decision has a strictly positive margin: each dispatch has a logit gap $\Delta = \ell_{k^*} - \max_{k \neq k^*} \ell_k \geq \Delta_{\min} > 0$, and each branch has a margin $m = |z| \geq m_{\min} > 0$.

2 Proofs

Theorem 1 (exact forward equivalence). *For every reachable state x and every finite α, T , $\Phi_S(x) = \Phi_H(x)$; hence $x_t^S = x_t^H$ for all $t \leq N$. The two modes differ only in the Jacobian.*

Proof. We first establish the one-step equality $\Phi_S(x) = \Phi_H(x)$ for every reachable state x , and then extend it to whole trajectories by induction on the step index.

Fix a reachable state x . In both modes the handler to execute is selected by the decoded opcode byte through the same hard switch (the softmax form (2) is never evaluated on the forward path), so both modes run the same handler on the same input. Within a handler the output is assembled from operations that agree value-for-value between the two modes. A read of the ROM or RAM at an integer address a is the one-hot dot product $\text{peek}(r, a) = \mathbf{1}_a^\top r$ (1); since a is an integer index the indicator $\mathbf{1}_a$ is exact, so $\text{peek}(r, a) = r_a$, the value a hard array read returns. Register, status-flag, binary-coded-decimal, and timer updates are computed with the same integer and round arithmetic in both modes (soft mode merely keeps the results in float32). A conditional branch returns $\text{STE}(\text{pc}_{\text{soft}}, \text{pc}_{\text{hard}}) = \text{pc}_{\text{soft}} + \text{sg}(\text{pc}_{\text{hard}} - \text{pc}_{\text{soft}})$ (4), which equals pc_{hard} for every α and T because sg is the identity in the forward direction. Each component of the output therefore equals its hard counterpart, and as the two handlers act on the same input their outputs coincide; hence $\Phi_S(x) = \Phi_H(x)$.

Now let the two runs share the initial state, $x_0^S = x_0^H$, with $x_{t+1}^\bullet = \Phi_\bullet(x_t^\bullet)$, and prove $x_t^S = x_t^H$ for all $0 \leq t \leq N$ by induction on t . The base case $t = 0$ holds by the shared initial state. For the inductive step, assume $x_t^S = x_t^H =: x$ for some $t < N$; then

$$x_{t+1}^S = \Phi_S(x_t^S) = \Phi_S(x) = \Phi_H(x) = \Phi_H(x_t^H) = x_{t+1}^H,$$

where the second and fourth equalities use the inductive hypothesis and the third is the one-step equality. By induction the trajectories coincide for all $t \leq N$.

Both modes thus compute the same real number at every step, and these are equal in float32 as well: IEEE-754 single precision has a 24-bit significand and so represents every integer in $[-2^{24}, 2^{24}]$ exactly (IEEE 2019; Goldberg 1991). Every VCS quantity lies in this range (data bytes ≤ 255 , 13-bit addresses, and per-frame cycle counts of a few tens of thousands), so the equality is exact rather than approximate.

Finally, the stop-gradient in the branch alters only the reverse-mode rule: it gives the returned value the derivative of pc_{soft} , the sigmoid-blended counter, which is generically nonzero where the hard branch—a step function of the flag—has zero or undefined derivative; the one-hot read likewise has Jacobian $\mathbf{1}_a$ with respect to r . The forward pass is unchanged while gradients become available. $\square$

Theorem 2 (temperature-limit bound). *For the fully relaxed variant (softmax dispatch and sigmoid branch, no straight-through correction), under Assumption (DM),*

$$\|\Phi_S^T(x) - \Phi_H(x)\|_\infty \leq C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}],$$

and over N steps with Lipschitz constant $L \geq 1$ the trajectory error is at most $\frac{L^N - 1}{L - 1}$ times this, which is $\mathcal{O}(e^{-c/T})$ with $c = \Delta_{\min}$.

Proof. We bound the deviation of the relaxed one-step map from the hard one and then propagate it along the trajectory. Throughout, C bounds the range of any state value (bytes ≤ 255 , branch offsets ≤ 256).

Consider first the dispatch. At a step with logits ℓ and winner k^* the gap is $\Delta \geq \Delta_{\min} > 0$ by Assumption (DM), so for any loser $k \neq k^*$,

$$w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}} \leq \frac{e^{\ell_k/T}}{e^{\ell_{k^*}/T}} = e^{(\ell_k - \ell_{k^*})/T} \leq e^{-\Delta_{\min}/T},$$

and the non-winning mass is $1 - w_{k^*} = \sum_{k \neq k^*} w_k \leq (K-1)e^{-\Delta_{\min}/T}$. Since the dispatch output is the convex combination $\sum_k w_k V_k$,

$$\begin{aligned} \left\| \sum_k w_k V_k - V_{k^*} \right\|_\infty &\leq (1 - w_{k^*}) \max_k \|V_k - V_{k^*}\|_\infty \\ &\leq (K-1)e^{-\Delta_{\min}/T} C. \end{aligned}$$

The sigmoid branch contributes similarly: for a margin $m = |z| \geq m_{\min}$ the gate error is $|\sigma(\alpha z) - \mathbb{I}[z > 0]| = \sigma(-\alpha m) \leq e^{-\alpha m_{\min}}$, so the blended counter $(1-g)\text{pc}_{\bar{b}} + g\text{pc}_b$ differs from the hard target by at most $|\text{pc}_b - \text{pc}_{\bar{b}}| e^{-\alpha m_{\min}} \leq C e^{-\alpha m_{\min}}$. Adding the two contributions bounds the per-step deviation: for every reachable x , $\delta := \|\Phi_S^T(x) - \Phi_H(x)\|_\infty \leq C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}]$.

To propagate it, let $L \geq 1$ be a Lipschitz constant of Φ_H in $\|\cdot\|_\infty$ and $e_t = \|x_t^{T,S} - x_t^H\|_\infty$ the trajectory error after t steps, with $e_0 = 0$. Each step adds a deviation of at most δ to the hard map applied to the accumulated error, giving $e_{t+1} \leq L e_t + \delta$; unrolling, $e_N \leq \delta \sum_{i=0}^{N-1} L^i = \delta (L^N - 1)/(L - 1)$. As $T \rightarrow 0$ (and $\alpha \rightarrow \infty$, since $m_{\min} = \Theta(1)$), $\delta = \mathcal{O}(e^{-\Delta_{\min}/T}) \rightarrow 0$, so the relaxed trajectory converges to the hard one and $\Phi_S^T \rightarrow \Phi_H$. $\square$

3 Effect of the Relaxation Parameters on the Gradient

Theorems 1 and 2 say the hard limit ($\alpha \rightarrow \infty$ for the sigmoid branch, $T \rightarrow 0$ for the softmax select) is exact in value but degenerate in gradient. Figure 1 shows this directly on jutari as the relaxation values are swept. We render a single player cannon whose horizontal position is produced by the soft primitives—a sigmoid-gated blend of a left and a right placement (`soft_branch`, sharpness α , top row) and a softmax mixture over candidate columns (`soft_select`, temperature T , bottom row)—and plot the screen-space directional-derivative saliency $\partial(\text{screen})/\partial(\text{control})$, that is, how each pixel’s value changes as the scalar control input increases. The colour encodes the *sign* of that derivative: red where a pixel brightens as the control increases (the sprite arriving) and blue where it darkens (the sprite leaving); the colour intensity is the magnitude, normalised to the row maximum, and the per-panel number is the peak magnitude relative to that maximum. Pixels with no sensitivity keep the faint grey of the rendered cannon.

For the branch the position is $\text{pc} = (1-g)\text{pc}_L + g\text{pc}_R$ with $g = \sigma(\alpha z)$, so its derivative with respect to the control z is $\alpha \sigma(\alpha z)(1 - \sigma(\alpha z))(\text{pc}_R - \text{pc}_L)$: a dipole, blue on the placement being vacated and red on the one being entered. The magnitude $\alpha \sigma(\alpha z)(1 - \sigma(\alpha z))$ is *non-monotonic* in α at the fixed operating point $z=0.25$. A small α gives a shallow gate and hence a small slope; a large α saturates the gate ($\sigma(\alpha z) \rightarrow 1$, so $\sigma(1 - \sigma) \rightarrow 0$). The maximum is the intermediate α for which αz falls in the sigmoid’s steep band, near $\alpha=6$ here— $\alpha=2, 6, 20$ give peak magnitudes 0.53, 1.00, 0.15, and at $\alpha=20$ the gate has essentially switched and the gradient has nearly vanished (the hard-branch limit).

For the select the rendered cannon is the softmax-weighted sum $\sum_k w_k$ (cannon at column k) with $w = \text{softmax}(\ell/T)$. At a high temperature the weights spread across several neighbouring candidate columns, so the render is a superposition of the cannon at those columns—the several faint copies seen at $T=2.0$ —and the gradient is spread over them. As T falls the weights concentrate: at $T=0.5$ only a few copies remain, and at $T=0.1$ the mixture is effectively one-hot, a single sharp cannon whose chosen column no longer moves under a small change of the control, so the gradient collapses to zero between candidate boundaries (peak magnitudes 1.00, 0.51, 0.00 for $T=2.0, 0.5, 0.1$).

In every panel the forward render is the exact hard one (Theorem 1); only the gradient changes. It is most informative at intermediate relaxation and decays to zero as the relaxation is sharpened toward the hard limit (Theorem 2). This is a qualitative study of how the parameter *values* act on the gradient, run on jutari; the values need not be tuned for the bit-exact forward pass.

4 Implementation Effort

Figure 2 is the evidence behind the effort estimate in the main paper. Because every step of the project was committed, the commit timestamps record when work actually happened. To turn them into an active-time figure we group the commits

$\partial(\text{screen})/\partial(\text{control})$ vs. relaxation strength (vanishes in the hard limit $\alpha \rightarrow \infty, T \rightarrow 0$)

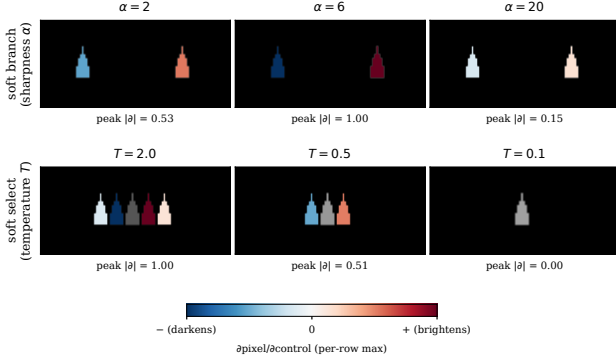


Figure 1: Screen-space gradient $\partial(\text{screen})/\partial(\text{control})$ on jutori as the relaxation strength is swept. **Top:** sigmoid branch (soft_branch), increasing sharpness α ; the gradient is a dipole between the left and right placements. **Bottom:** softmax select (soft_select), decreasing temperature T ; a high T renders the cannon as several weighted copies. Colour gives the sign of the per-pixel derivative (red: the pixel brightens as the control increases; blue: it darkens) and its intensity the magnitude, normalised to the row maximum (the colour bar is in these per-row units); the per-panel number is the peak relative to that maximum, and faint grey is the rendered cannon where the gradient is negligible. The gradient is largest at intermediate relaxation and vanishes toward the hard limit (large α , small T), consistent with Theorems 1 and 2.

into sessions, treating a gap of more than three hours between consecutive commits as a session boundary. Three hours is well above the in-session rhythm—the median gap between consecutive commits is about 13 minutes—and well below the multi-hour and overnight gaps that separate genuine work periods, so the partition is insensitive to the exact threshold: the active total is 106 hours at a two-hour cutoff and 162 hours at a four-hour cutoff, bracketing the 137 hours obtained at three hours.

With the three-hour boundary the 373 commits fall into 52 sessions whose durations sum to 136.8 hours, i.e. about 5.7 eight-hour working days, even though they are spread over a 29-day calendar window. The remaining $\sim 80\%$ of the calendar span is idle and excluded; it consists mostly of stretches during which the lead programmer was travelling and without reliable internet access, which appear in the plot as the long flat segments. This active total—not the calendar span—is what we compare against the scale of the reference codebase in the main paper.

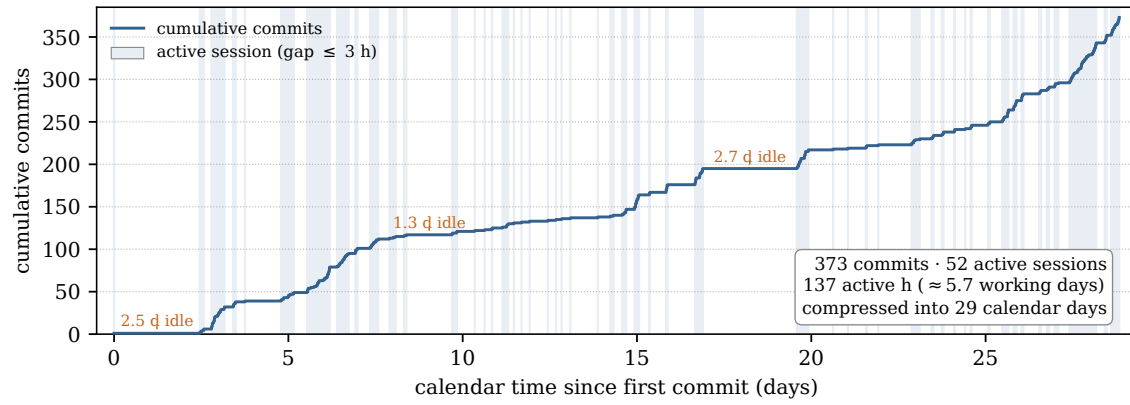


Figure 2: Implementation effort reconstructed from the version-control log. The step curve is the cumulative number of commits against calendar time; each shaded band is an active work session (a maximal run of consecutive commits no more than three hours apart), and the long flat stretches between bands are idle gaps, the largest of which are annotated. The 373 commits form 52 active sessions totalling ~ 137 hours of work—about 5.7 working days—spread across a 29-day calendar window.

References

- Goldberg, D. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, 23(1): 5–48.
- IEEE. 2019. IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019). IEEE Std 754-2019.